

JobHunter v2

Complete System Guide

Architecture, components, security, operations,
v2 change log, deployment, and verification

4

public job APIs

3

ingestion paths

1

shared matching engine

PDF

cover letters in Telegram

At a glance

AI interprets. Python decides. JobHunter v2 expands discovery to four public APIs and creates a tailored PDF only when an APPLY posting explicitly requires a cover letter.

Layer	Current implementation
Ingestion	Manual API, four scheduled public APIs, authenticated TMU browser flow
Processing	API Gateway, Lambda, SQS, structured AI extraction, deterministic matching
Data	S3 resume, DynamoDB results, resume cache, and seen-job dedupe
Delivery	Telegram APPLY notifications with optional attached cover-letter PDF
Operations	CloudWatch logs and alarms, SNS alerts, DLQ, Terraform

1. Executive summary

JobHunter is an event-driven serverless job discovery and resume-matching platform. It collects postings from public job APIs, a manual API, and the authenticated Toronto Metropolitan University co-op portal. Each posting is normalized, queued, analyzed against a stored resume, scored by deterministic Python rules, saved in DynamoDB, and conditionally delivered through Telegram.

The central design rule is:

AI interprets.
Python decides.

AI turns unstructured job and resume text into constrained structured data. Python applies the scoring policy and returns APPLY, STRETCH, or SKIP.

JobHunter v2 adds:

1. Multi-source discovery from Remotive, Jobicy, Remote OK, and The Muse.
2. Source attribution and canonical application links throughout the pipeline.
3. Explicit cover-letter requirement detection with exact evidence.
4. Tailored cover-letter generation for qualifying APPLY jobs.
5. In-memory PDF creation and Telegram document delivery.

2. Detailed v2 engineering log

2.1 Why more ingestion sources were needed

The original scheduled fetcher used only Remotive. It was reliable but produced too few useful Canadian co-op and entry-level infrastructure roles. The v2 fetcher keeps Remotive and adds three authorized public APIs.

Provider	JobHunter query scope	Authentication	Link handling
Remotive	Software development	None	Canonical URL preserved
Jobicy	Remote jobs eligible for Canada	None	Canonical URL preserved
Remote OK	Development, Python, cloud, and DevOps tags	None	Application URL preferred
The Muse	Entry-level Toronto jobs	None for current use	Landing-page URL preserved

Each provider has a different response schema. Provider adapters normalize every listing to:

```
{
  "title": "Cloud Engineering Co-op",
  "company": "Example Company",
  "url": "https://example.com/apply",
  "job_text": "Full posting text",
  "source": "jobicy"
}
```

This keeps SQS, the worker, DynamoDB, and Telegram independent from provider-specific fields.

2.2 Concurrent provider fetching

Four serial HTTP calls could consume most of the fetcher Lambda's 60-second timeout. `fetch_all_jobs()` now uses `ThreadPoolExecutor` to call all providers concurrently.

Each provider call has isolated exception handling. If one API is slow or unavailable, the remaining providers still produce jobs. This makes total wait time close to the slowest provider rather than the sum of all four.

2.3 Fair source interleaving

The existing limit of five new relevant jobs per scheduled run remains in place to control AI cost. A simple concatenated list could allow the largest provider to occupy all five slots.

Jobs are now interleaved in round-robin order:

```
Remotive -> Jobicy -> Remote OK -> The Muse -> repeat
```

Entry-level titles are prioritized inside each provider before interleaving. This lets every source contribute without raising the daily processing budget.

2.4 Cross-source and persistent deduplication

The fetcher removes duplicate company-and-title pairs found across APIs during the current run. The `jobhunter-seen-jobs` DynamoDB table continues to prevent already accepted public postings from being processed again on later runs.

This avoids paying to analyze syndicated copies of the same employer posting.

2.5 Source and URL propagation

The previous Remotive adapter read a URL but dropped it when constructing the SQS message. v2 passes metadata through the complete route:

```
Provider -> Fetcher -> SQS -> Worker -> DynamoDB -> Telegram
```

The queue message now carries `job_id`, `title`, `company`, `job_text`, `source`, and `url`. The manual and TMU API paths also accept an optional URL. DynamoDB stores it as `job_url`, and Telegram shows the source and application link.

2.6 Cover-letter requirement detection

The structured job analysis now contains:

```
requires_cover_letter: bool
cover_letter_evidence: str
```

The model may return `true` only when the posting explicitly asks the applicant to submit, upload, or include a cover letter. It must supply a short exact excerpt as evidence. When no request exists, the value is `false` and the evidence is empty.

This prevents unnecessary generation and makes the decision auditable.

2.7 Tailored letter generation

The worker generates a letter only when both conditions are true:

```
recommendation == APPLY
requires_cover_letter == true
```

The generator receives the stored resume, full posting, title, and company. Its rules require it to use only supported resume facts, avoid invented experience or metrics, connect strong evidence to the posting, use an early-career professional tone, and remain about 220 to 300 words.

The result begins with `Dear Hiring Team`, and ends with `Sincerely, Raihan`.

Letter generation has its own error boundary. If it fails, the match is still stored successfully and the error is logged. An optional document cannot invalidate the primary matching result.

2.8 In-memory PDF generation

`pdf_document.py` builds a PDF from standard-library code. It provides paragraph preservation, word wrapping, margins, PDF text escaping, Helvetica text, valid cross-reference data, and automatic multipage output.

No PDF file is required on Lambda disk and no large PDF dependency was added to the deployment package. This keeps packaging and cold-start overhead low.

2.9 Telegram document delivery

Two delivery paths now exist:

```
APPLY without requested letter -> Telegram sendMessage
APPLY with generated letter    -> Telegram sendDocument
```

The document route builds a multipart request and attaches `tailored-cover-letter.pdf`. Its caption contains the title, company, source, recommendation, match scores, application link, and decision reasons. The job and document therefore remain together in one Telegram message.

2.10 Worker timeout

The worker timeout increased from 60 to 120 seconds. A cover-letter job can require S3 retrieval, structured job analysis, deterministic matching, a second model request, DynamoDB storage, and a Telegram upload.

2.11 Deployment defect and repair

The first deployed end-to-end test reached an APPLY decision and generated the letter, but DynamoDB rejected the final update:

```
Invalid UpdateExpression: Attribute name is a reserved keyword;  
reserved keyword: source
```

The update expression now uses `#source`, mapped through `ExpressionAttributeNames` to the real `source` attribute. After redeploying the worker, a new end-to-end submission completed and the PDF arrived in Telegram.

This incident also proved that CloudWatch and SNS monitoring worked: the worker alarm changed to ALARM and sent the configured email notification.

3. Complete updated architecture



Operational support:

CloudWatch Logs and alarms -> visibility and failure detection
 SNS -> infrastructure email alerts
 SSM Parameter Store -> OpenAI, Telegram, and TMU secrets
 SQS DLQ -> repeatedly failed messages
 Terraform -> infrastructure and deployments

4. Component guide

4.1 API Gateway

Base endpoint: `https://t0r6wiqif4.execute-api.us-east-2.amazonaws.com`

Route	Purpose
POST /match	Submit a manual job asynchronously
GET /results/{job_id}	Retrieve processing or final status
POST /tmu-jobs	Receive authorized co-op jobs from localhost

4.2 API Lambda - jobhunter-lambda

File: `lambda_api_package/lambda_function.py`

It parses requests, validates bodies, authenticates TMU submissions, writes initial PROCESSING records, creates job IDs, performs atomic TMU deduplication, sends SQS messages, and returns saved results.

Manual jobs use UUIDs. TMU jobs use stable IDs in the form `tmu-{postingId}`. The TMU write uses `attribute_not_exists(job_id)` so concurrent duplicates cannot both be accepted.

4.3 EventBridge

EventBridge invokes the public-source fetcher once per day. This provides automated discovery without an always-on server.

4.4 Fetcher Lambda - jobhunter-fetcher

File: `lambda_fetcher_package/fetcher_function.py`

The fetcher calls four providers, normalizes schemas, prioritizes entry-level roles, interleaves sources, removes batch duplicates, rejects senior titles, filters for target roles, checks persistent dedupe, creates initial results, and queues at most five new relevant jobs per run.

Strong target terms include DevOps, cloud, infrastructure, platform, SRE, and AWS. Secondary terms include QA, test, support, systems, and service desk.

4.5 TMU Chrome extension

Files: `tmu_extension/manifest.json`, `content.js`, and `background.js`.

The extension operates inside the user's normal authenticated session. It inspects visible rows, filters relevant jobs, fetches details through the browser session, extracts posting text, sends normalized data to localhost, retries once when expected detail markers are missing, and records successfully accepted posting IDs in Chrome storage.

It does not store credentials or transmit cookies, CSRF values, or opaque session values to AWS.

4.6 Local TMU collector

Files: `tmu_collector/collector_server.py` and `submit_job.py`.

The collector binds to `127.0.0.1:8765`. It accepts normalized extension data, retrieves the TMU ingest key from SSM through AWS CLI, and forwards the posting to `POST /tmu-jobs` with `x-jobhunter-key`.

4.7 Amazon SQS

`jobhunter-match-queue` decouples producers from AI processing. `jobhunter-match-dlq` receives messages that fail repeatedly. The 360-second visibility timeout is longer than the worker's 120-second timeout, reducing concurrent duplicate processing.

4.8 Worker Lambda - `jobhunter-worker`

File: `lambda_worker_package/worker_function.py`

For every SQS message, the worker loads the S3 resume, checks the resume cache, analyzes the posting, runs the matcher, preserves metadata, optionally generates a letter, updates DynamoDB, and notifies Telegram for APPLY results.

Unexpected failures mark the record `FAILED` when possible and are re-raised so SQS retry and DLQ behavior remains active.

4.9 AI extractor

File: `lambda_worker_package/ai_extractor.py`

The extractor uses a controlled skill vocabulary and Pydantic schema. It identifies title, minimum experience, required skills, preferred skills, mentioned skills, other requirements, and cover-letter requirements. Each technical skill includes strength, reasoning, and exact evidence.

Python validation discards unknown skills, invalid strength values, and duplicates across categories.

4.10 Resume cache

The worker reads `s3://jobhunter-resume-storage/resume.txt`, calculates its SHA-256 hash, and queries `jobhunter-resume-cache`. A hit reuses the previous structured resume analysis. A miss performs analysis and stores it.

Caching removes one repeated model request from nearly every job.

4.11 Deterministic matcher

File: `lambda_worker_package/matcher.py`

The matcher evaluates evidence strength, required experience, related skill families, project evidence, missing hard requirements, and missing strong requirements. It computes technical and experience percentages and produces readable reasons.

Python, rather than the model, chooses the final recommendation. This makes decisions repeatable and auditable.

4.12 Cover-letter generator

File: `lambda_worker_package/cover_letter.py`

This module performs the second model request only when necessary. The final letter is stored as text in the DynamoDB result before delivery.

4.13 PDF generator

File: `lambda_worker_package/pdf_document.py`

It converts the letter to PDF bytes in memory and can create multiple pages. The normal 220-to-300-word target generally fits on one page.

4.14 DynamoDB

Table	Purpose
<code>jobhunter-results</code>	Processing state, match details, metadata, and letters
<code>jobhunter-resume-cache</code>	Resume analysis keyed by resume hash
<code>jobhunter-seen-jobs</code>	Public-source deduplication

The v2 result stores `source`, `job_url`, `requires_cover_letter`, `cover_letter_evidence`, and `cover_letter` in addition to the existing scores and reasons.

4.15 SSM Parameter Store

Known parameters:

```
/jobhunter/openai-api-key
/jobhunter/telegram-bot-token
/jobhunter/telegram-chat-id
/jobhunter/tmu-ingest-key
```

Secret values never belong in Git, documentation, test output, or chat messages.

4.16 CloudWatch, SNS, and DLQ

CloudWatch captures Lambda output, token usage, timing, cache behavior, provider failures, match details, and Telegram status. Alarms monitor worker errors, fetcher errors, and DLQ depth. SNS emails infrastructure alerts, while Telegram delivers useful job results.

4.17 Terraform

File: `terraform/main.tf`

Terraform packages Lambda source folders and manages functions, IAM roles, API integration, queues, tables, storage, the EventBridge schedule, alarms, SNS, and SSM permissions.

The initial v2 deployment plan was 0 to add, 3 to change, 0 to destroy. It updated the API, fetcher, and worker code in place. The reserved-word repair required a worker-only update.

5. End-to-end flows

Scheduled API flow

```
EventBridge -> four APIs -> normalize -> interleave -> filter -> dedupe
-> PROCESSING record -> SQS -> worker -> match -> result -> Telegram
```

TMU flow

```
Normal browser login -> extension -> localhost collector -> secure API  
-> atomic dedupe -> SQS -> shared worker -> same letter and PDF behavior
```

The cover-letter feature applies to TMU because it lives in the shared worker. TMU may not supply a stable clickable application URL because its portal uses authenticated session-dependent navigation.

Manual flow

```
POST /match -> PROCESSING -> SQS -> worker -> GET /results/{job_id}
```

6. Security boundaries

- OpenAI, Telegram, and TMU secrets live in SSM.
- Lambda functions use separate least-privilege IAM roles.
- TMU authentication remains in normal Chrome.
- Credentials, cookies, CSRF values, and browser profiles are never sent to AWS.
- JobHunter does not bypass Cloudflare, CAPTCHA, authentication, or anti-bot controls.
- Only normalized posting data crosses the browser-to-localhost boundary.
- Terraform state, deployment ZIPs, resumes, .env, virtual environments, and test payloads remain uncommitted.

7. Verification record

Provider tests

Live normalization returned 200 Jibcity jobs, 31 Remote OK jobs, and 20 The Muse jobs. The complete combined test returned 267 listings and began in correct round-robin order.

Static checks

- All affected Python files passed syntax parsing.
- `git diff --check` passed.
- Terraform validation passed.
- New source modules were explicitly unignored.

Detection tests

An explicit sentence requesting a resume and cover letter returned `True` plus the exact evidence. A posting requesting only a resume returned `False` and empty evidence.

PDF tests

- Sample PDF opened correctly in the local viewer.
- A real tailored letter was generated using the stored resume.
- Programmatic parsing confirmed valid single-page and multipage PDFs.

- Extracted text matched the original input.

Telegram test

A local document upload returned HTTP 200. Telegram showed the caption, link, and attached PDF.

Full AWS test

A controlled posting entered through the deployed API and produced:

```
Recommendation: APPLY
Technical match: 69.98%
Experience match: 100%
Cover letter requested: true
```

The first run exposed the DynamoDB reserved-word issue and triggered the worker alarm. After repair and redeployment, a fresh submission completed and delivered its PDF through Telegram.

8. Failure behavior

Failure	System behavior
One provider fails	Other providers continue
Job analysis fails	Worker fails and SQS retries
Letter generation fails	Match remains valid; letter remains empty
DynamoDB update fails	Worker re-raises; alarm and retry behavior activate
Telegram fails	Stored result remains complete; error is logged
Message repeatedly fails	SQS moves it to the DLQ

9. Repository map

```
jobhunter/  
|-- README.md  
|-- JOBHUNTER_V2_ENGINEERING_LOG_AND_SYSTEM_GUIDE.md  
|-- lambda_api_package/lambda_function.py  
|-- lambda_fetcher_package/fetcher_function.py  
|-- lambda_worker_package/  
|   |-- worker_function.py  
|   |-- matcher.py  
|   |-- ai_extractor.py  
|   |-- resume_extractor.py  
|   |-- cover_letter.py  
|   |-- pdf_document.py  
|   |-- skills.py  
|   |-- requirements.txt  
|-- terraform/main.tf  
|-- tmu_collector/  
|   |-- collector_server.py  
|   |-- submit_job.py  
|-- tmu_extension/  
|   |-- manifest.json  
|   |-- content.js  
|   |-- background.js
```

10. Cost and scaling

The main variable cost is AI usage. Controls include a five-job scheduled limit, title filtering before AI, senior-role rejection, persistent dedupe, cross-source dedupe, resume caching, and letter generation only for qualifying APPLY jobs.

The AWS architecture is serverless and usage-based. If volume increases, the main planning areas are queue concurrency, provider rate limits, AI budget, DynamoDB retention, and Telegram volume.

11. Known limitations

- Only five new relevant scheduled jobs are processed per run.
- External API availability and schemas can change.
- The Muse currently targets entry-level Toronto results.
- Remote listings can still require manual location-eligibility review.
- TMU may not provide a stable public application link.
- Generated letters must be reviewed before submission.
- PDFs are delivered but not stored permanently in S3.
- PDF styling is intentionally clean and minimal.

12. Recommended next improvements

1. Add provider fixture tests and Telegram multipart tests.
 2. Add configurable provider-specific limits.
 3. Add Adzuna as an optional credentialed Canadian source.
 4. Add custom CloudWatch metrics for fetch, filter, queue, match, and letter counts.
 5. Add a dashboard for results, filters, links, and stored letters.
 6. Optionally store generated PDFs in a private S3 prefix.
 7. Add CI checks for Python, Terraform, fixtures, and deployment-package contents.
-

13. Completion state

Manual ingestion	COMPLETE
Remotive ingestion	COMPLETE
Jobicy ingestion	COMPLETE
Remote OK ingestion	COMPLETE
The Muse ingestion	COMPLETE
Concurrent fetching	COMPLETE
Fair source interleaving	COMPLETE
Source and application links	COMPLETE
TMU authenticated ingestion	COMPLETE
Resume caching	COMPLETE
Deterministic matching	COMPLETE
Cover-letter detection	COMPLETE
Grounded letter generation	COMPLETE
In-memory PDF creation	COMPLETE
Telegram PDF attachment	COMPLETE
DynamoDB persistence	COMPLETE
CloudWatch/SNS/DLQ monitoring	COMPLETE
Terraform deployment	COMPLETE
End-to-end production test	PASSED

At the time of this document, v2 is deployed and operating. The local source changes still require normal Git review, commit, and push if they have not already been recorded.

14. External references

- Remotive: <https://remotive.com/api/remote-jobs>
 - Jobicy: <https://github.com/Jobicy/remote-jobs-api>
 - Remote OK: <https://remoteok.com/api>
 - The Muse: <https://www.themuse.com/developers/api/v2>
 - Telegram Bot API: <https://core.telegram.org/bots/api>
 - AWS Lambda: <https://docs.aws.amazon.com/lambda/>
 - Amazon SQS: <https://docs.aws.amazon.com/sqs/>
 - Amazon DynamoDB: <https://docs.aws.amazon.com/dynamodb/>
-

15. Final system principle

```
Sources discover.  
Adapters normalize.  
SQS decouples.  
AI interprets.  
Python decides.  
DynamoDB records.  
Telegram delivers.  
CloudWatch observes.  
Terraform reproduces.
```

This separation lets JobHunter add new sources and documents without rebuilding the matcher or weakening the TMU security boundary.